23CSE202 DATABASE MANAGEMENT SYSTEMS
S3 B.Tech CSE
Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

PROJECT REPORT

Group No :

Team Members :

Roll No	Student Name
AM.SC.U4CSE25245	Saharsh Baiju
AM.SC.U4CSE25244	Rishik Raj P
AM.SC.U4CSE25212	Dev Narayan U
AM.SC.U4CSE25227	Kashyap S

Project Title : **O(1): An ICPC Practice Tracker for College Teams**

CONTENTS

Sl. No	Chapter	Page No
1	Abstract	3
2	Introduction	4
3	Requirements (Functional & Non-functional)	5
4	UI Design	6
5	Database Design	7
6	Database Selection & Justification	10
7	Implementation	11
8	Testing & Validation	14
9	Results & Screenshots	16
10	Challenges Faced & Solutions	20
11	Conclusion & Future Enhancements	21
	References	22

1. Abstract

ICPC teams in our college practise on Codeforces and track their progress in a shared spreadsheet. The spreadsheet repeats the same student and problem details on every row, so it is hard to update and often goes wrong. We built **O(1)**, a web app backed by a PostgreSQL database of nine tables. Students sign up, link their Codeforces handle and import all their submissions with one click. They can form teams of up to three, make shared practice sheets, and compare each other on a leaderboard and a team analytics page. We designed the tables using functional dependencies, closures, minimal cover and normalisation up to BCNF, and checked that the split is lossless and dependency preserving. The database uses keys, foreign keys, CHECK constraints, a trigger, views, an index and transactions. Every number shown in the app comes from a SQL query, so it always matches the stored data.

2. Introduction

2.1 Background

ICPC is a contest where teams of three solve programming problems. Teams prepare on Codeforces, which has thousands of problems with topics (DP, graphs, greedy, ...) and ratings from 800 to 3500. A team needs to know what each member has solved, practise the same problems together, and find its weak topics.

2.2 Existing system and its problems

- **Codeforces shows one person at a time.** There is no team or college view.
- **The spreadsheet repeats data.** Changing a team name means editing many rows (update anomaly). A problem cannot be added until someone tries it (insertion anomaly). Deleting a student's only attempt also deletes the problem (deletion anomaly).
- **Everything is typed by hand,** so it is slow and full of mistakes.
- **No rules.** Nothing stops a fourth team member or the same college being typed two different ways.

2.3 Our solution

O(1) keeps the same data in a normalised database and builds every page from SQL queries. Submissions are imported from Codeforces, so nobody types them. Teams, colleges and sheets are stored only in our database, because Codeforces has no such data. Counts and ranks are calculated by views, not stored, so they are always correct. Rules like "at most 3 members" are enforced by the database itself.

2.4 Objectives

1. Design a schema in BCNF from the old spreadsheet using FDs, closures and a minimal cover.
2. Prove the design is lossless and dependency preserving.
3. Build it in PostgreSQL with constraints, a trigger, views and an index.
4. Build a web app (React + Flask) where every feature uses SQL.
5. Give teams shared sheets and side-by-side analytics.

3. Requirements (Functional & Non-functional)

3.1 Functional requirements

Table 1: Functional requirements

ID	Requirement	SQL used
FR1	Sign up with name, e-mail, password, college and Codeforces handle. Suggest an existing college while typing.	INSERT, fuzzy search
FR2	Log in and log out.	SELECT
FR3	Edit profile and change password.	UPDATE
FR4	Import all submissions from Codeforces. Importing again adds no duplicates.	INSERT ... ON CONFLICT
FR5	Search problems by code, title, topic and difficulty.	JOIN, GROUP BY, view
FR6	See submission history for me, my team or everyone.	JOIN, WHERE
FR7	Create, join and leave a team (max 3 members).	INSERT, UPDATE, trigger
FR8	Create and delete team practice sheets from any Codeforces problem.	INSERT, DELETE CASCADE
FR9	Track each member's progress on each sheet.	view, NOT EXISTS
FR10	Leaderboard, filter by college.	view, rank() OVER
FR11	Team analytics: rating, solved, streaks, topics.	CTE, window functions
FR12	Database page showing live tables (for the viva).	SELECT *

3.2 Non-functional requirements

- **Integrity:** all rules are in the schema, so they hold even if the app has a bug.
- **Security:** passwords are stored as scrypt hashes; all values are passed as SQL parameters, so SQL injection is not possible.
- **Reliability:** each request is one transaction, so it fully happens or not at all.
- **Performance:** pages use a few indexed queries; Codeforces is called only when syncing.
- **Usability:** simple black-and-white design that works on phones and laptops.
- **Software:** PostgreSQL 14+, Python 3.10+ with Flask and psycopg2, Node.js with React, any browser.

4. UI Design

The app has a sidebar with all pages and a top bar with problem search. Every data page has an **SQL** button that shows the query behind it. The flow is:

```
Landing -> Sign up / Log in -> Dashboard
    Dashboard    : my numbers, next problems, sheet progress, recent submissions
    Leaderboard  : students ranked by problems solved
    Problems     : search and filter, open on Codeforces
    Submissions  : history for me / team / everyone
    Sheets       : team practice lists, "New sheet" dialog
    My team      : create / join / leave, analytics, sync team
    Database     : live tables, views and DDL
    Profile      : edit details, change password
```

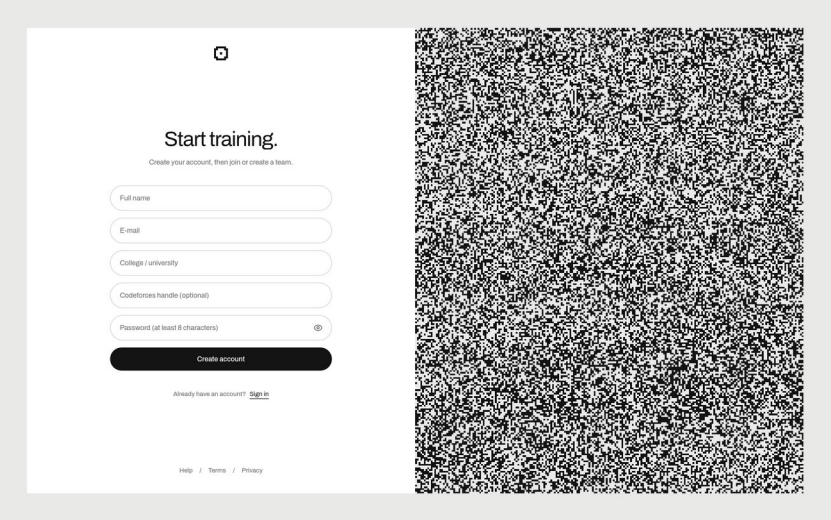


Figure 1: Sign-up page. The college box suggests colleges already in the database.

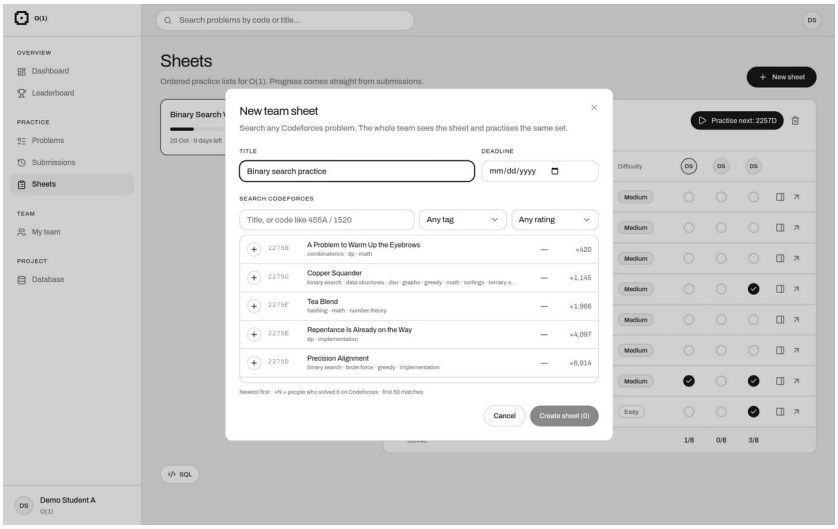


Figure 2: New sheet dialog. It searches all Codeforces problems; picked problems become rows of sheet_problem.

5. Database Design

5.1 ER diagram

Submission is a weak entity: it is identified by the user, the problem and the time. Difficulty is a derived attribute, so it is not stored.

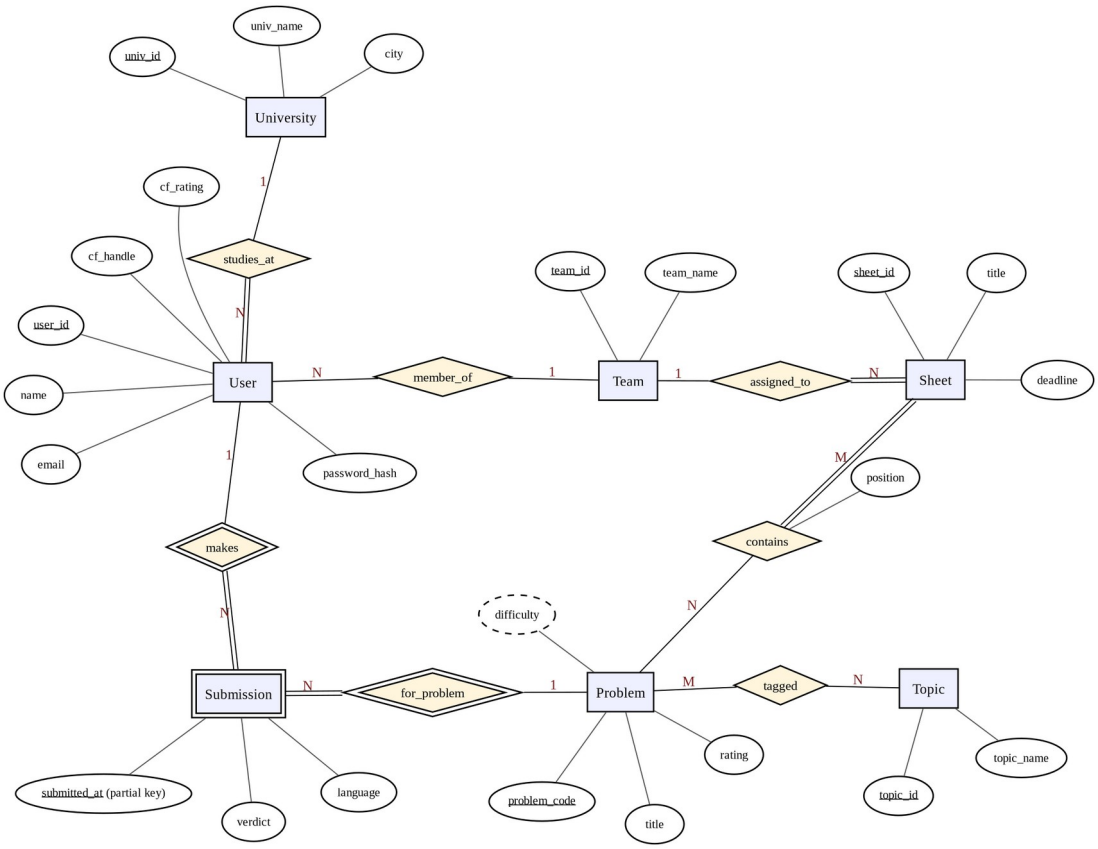


Figure 3: ER diagram of O(1).

Table 2: Relationships

Relationship	Entities	Cardinality	Note
studies_at	User – University	N : 1	every user has a college
member_of	User – Team	N : 1	optional, max 3 per team
assigned_to	Sheet – Team	N : 1	
contains	Sheet – Problem	M : N	with position
tagged	Problem – Topic	M : N	
makes / for_problem	User / Problem – Submission	1 : N	identifying (weak entity)

5.2 ER to tables

Each entity became a table. 1:N relationships became foreign keys on the N side (team_id is nullable because a user may have no team). M:N relationships became junction tables (problem_topic, sheet_problem). The weak entity Submission got the key (user_id, problem_code, submitted_at).

```
university    (univ_id PK, univ_name UNIQUE, city)
team          (team_id PK, team_name UNIQUE)
app_user      (user_id PK, name, email UNIQUE, password_hash, cf_handle UNIQUE,
               cf_rating, cf_max_rating, univ_id FK, team_id FK NULL)
topic         (topic_id PK, topic_name UNIQUE)
problem       (problem_code PK, title, rating CHECK 800-3500)
problem_topic (problem_code FK, topic_id FK)                    PK = both
submission    (user_id FK, problem_code FK, submitted_at, verdict CHECK, language)
                                                                PK = first three
sheet         (sheet_id PK, title, team_id FK, deadline)
sheet_problem (sheet_id FK, problem_code FK, position CHECK > 0)
                     PK = (sheet_id, problem_code), UNIQUE (sheet_id, position)
```

Constraints: NOT NULL and primary keys on every table; UNIQUE on e-mail, handle and names; CHECK on rating, verdict and position; ON DELETE CASCADE for owned rows (a user's submissions, a sheet's problems); ON DELETE SET NULL so deleting a team keeps its members; a trigger for the 3-member limit; a GIN trigram index for college search.

5.3 Normalisation

We started from our old spreadsheet, which had one row per submission and a list of topics in one cell. To keep the working short we name each column with a letter:

```
U user_id   N name   M email   I univ_id   J univ_name   G team_id   K team_name
P problem_code   T title   R rating   S submitted_at   V verdict   L language
```

1NF: the topics cell had many values, so we moved topics to Problem_Topic (one row per problem and topic) and a Topic table. Now every cell has one value.

Functional dependencies (F): U → NMIG, M → U, I → J, G → K, P → TR, UPS → VL, plus three extra ones the team wrote down: UM → N, U → J, UPS → T.

Closure and keys: $\{U,P,S\}^+$ gives all 13 columns (U adds N,M,I,G; then J and K; P adds T,R; UPS adds V,L). $\{M,P,S\}^+$ also gives all columns because M → U. No smaller set works, since P and S are never on the right side of any FD. So the candidate keys are {U,P,S} and {M,P,S}.

F^+ and membership: using Armstrong's axioms we get, for example, U → J (U → I, I → J) and M → K (M → U → G → K). To test X → Y we check if Y is in X^+: M → K is in F^+, but I → U is not, since $\{I\}^+ = \{I, J\}$.

Minimal cover: split right sides; UM → N reduces to U → N and UPS → T reduces to P → T (already there); U → J is redundant because U → I → J. Result:

```
Fc = { U → NMIG,   M → U,   I → J,   G → K,   P → TR,   UPS → VL }
```

2NF: U → NMIG and P → TR depend on only part of the key {U,P,S}. We split them out:

```
USER_INFO(U,N,M,I,J,G,K)      PROBLEM(P,T,R)      SUBMISSION(U,P,S,V,L)
```

3NF: in USER_INFO, I → J and G → K are transitive (U → I → J). We split again:

```
USER(U,N,M,I,G)      UNIVERSITY(I,J)      TEAM(G,K)
```

BCNF: now the left side of every FD is a key in its table, so every table is in BCNF. The other tables (topic, problem_topic, sheet, sheet_problem) also have only keys as determinants.

Lossless join: each split shares a column that is a key of one side (U, P, I, G), so joining back gives exactly the original rows. The chase test ends with the SUBMISSION row all 'a', which confirms it. If we had split SUBMISSION into (U,P) and (P,S,V,L), joining on P would create fake submissions.

Dependency preservation: every FD in Fc fits inside one table (I → J in UNIVERSITY, P → TR in PROBLEM, and so on), so each rule can be checked without joins.

Derived values: difficulty depends on rating (rating → difficulty), so storing it would break 3NF. We calculate it in the view v_problem_difficulty instead. Solved counts and sheet progress are also views.

6. Database Selection & Justification

We used **PostgreSQL 18**. Our data is connected (users, teams, problems, sheets), and our features need joins and grouping across many users, so a relational database fits best. We chose PostgreSQL because:

- It supports everything we needed: CTEs, window functions, the FILTER clause, views, CHECK constraints and INTERSECT.
- PL/pgSQL triggers let us enforce the 3-member rule.
- The pg_trgm extension gives fuzzy search for college names.
- INSERT ... ON CONFLICT makes the Codeforces import safe to repeat.
- It is free, open source, reliable (ACID transactions) and works well with Python (psycopg2).

Table 3: Comparison of databases

Feature	PostgreSQL	MySQL	SQLite	MongoDB
Window functions, CTEs	Yes	Yes	Yes	Partly
Triggers	Yes	Yes	Limited	No
Fuzzy text search	pg_trgm	No	No	Paid add-on
FILTER clause	Yes	No	Yes	No
Multi-user server	Yes	Yes	No	Yes
Cost	Free	Free	Free	Free

7. Implementation

7.1 How it works

The React front end calls our Flask back end (for example /api/sheets). Flask runs plain SQL on PostgreSQL using psycopg2 and sends back JSON. We did not use an ORM, so every query is easy to read. The schema is in schema-lite.sql and is created with one command: python init_db.py.

7.2 Tables, trigger and view

```
CREATE TABLE submission (
  user_id       INT         REFERENCES app_user(user_id) ON DELETE CASCADE,
  problem_code  VARCHAR(12) REFERENCES problem(problem_code),
  submitted_at  TIMESTAMP   NOT NULL,
  verdict       VARCHAR(25) NOT NULL CHECK (verdict IN ('OK', 'WRONG_ANSWER', ...)),
  language      VARCHAR(50) NOT NULL,
  PRIMARY KEY (user_id, problem_code, submitted_at)
);

CREATE FUNCTION check_team_size() RETURNS trigger AS $$
BEGIN
  IF NEW.team_id IS NOT NULL AND
     (SELECT count(*) FROM app_user
       WHERE team_id = NEW.team_id AND user_id <> NEW.user_id) >= 3 THEN
    RAISE EXCEPTION 'team % already has 3 members', NEW.team_id;
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER team_size_check BEFORE INSERT OR UPDATE OF team_id ON app_user
  FOR EACH ROW EXECUTE FUNCTION check_team_size();

CREATE VIEW v_problem_difficulty AS
SELECT problem_code, title, rating,
       CASE WHEN rating IS NULL THEN NULL WHEN rating < 1400 THEN 'Easy'
            WHEN rating < 2000 THEN 'Medium' ELSE 'Hard' END AS difficulty
FROM problem;
```

7.3 Sample data

The app has no built-in data; it comes from sign-up and the Codeforces sync. For testing we made four demo accounts linked to public Codeforces handles. After one sync the database had 26,944 submissions, 9,591 problems, 21,535 problem–topic rows and 37 topics. The import uses ON CONFLICT DO NOTHING, so syncing again adds nothing twice:

```
INSERT INTO submission (user_id, problem_code, submitted_at, verdict, language)
VALUES (%s, %s, %s, %s, %s)
ON CONFLICT (user_id, problem_code, submitted_at) DO NOTHING;
```

7.4 CRUD operations

Table 4: CRUD examples

Operation	Feature	SQL
Create	Sign up	INSERT INTO app_user (...) VALUES (...) RETURNING user_id
Read	Leaderboard	SELECT rank() OVER (ORDER BY solved DESC), ... FROM v_leaderboard
Update	Join team	UPDATE app_user SET team_id = %s WHERE user_id = %s
Delete	Delete sheet	DELETE FROM sheet WHERE sheet_id = %s (its problems are deleted by CASCADE)

Values are always passed as %s parameters, never joined into the SQL text, which stops SQL injection. Each request is one transaction: if any statement fails, everything is rolled back.

7.5 Queries with output

All outputs below are from our demo data. The full list of queries is in the separate SQL document.

Q1. Join of three tables: latest accepted submissions

```
SELECT u.name, p.problem_code, p.title, s.submitted_at
FROM submission s
JOIN app_user u ON u.user_id = s.user_id
JOIN problem  p ON p.problem_code = s.problem_code
WHERE s.verdict = 'OK'
ORDER BY s.submitted_at DESC
LIMIT 5;
```

name	problem_code	title	submitted_at
Demo Student A	2271H1	Personal Issues (Easy Version)	2026-10-10 18:38
Demo Student A	2271F	Beautiful Pillers	2026-10-10 17:21
Demo Student B	2271H1	Personal Issues (Easy Version)	2026-10-10 16:50
Demo Student B	2271F	Beautiful Pillers	2026-10-10 16:19

(4 of 5 rows shown)

Q2. GROUP BY with HAVING: students with 2,000+ solved

```
SELECT u.name, count(DISTINCT s.problem_code) AS solved
FROM app_user u JOIN submission s ON s.user_id = u.user_id
WHERE s.verdict = 'OK'
GROUP BY u.name
HAVING count(DISTINCT s.problem_code) >= 2000;
```

name	solved
Demo Student B	4596
Demo Student C	3956
Demo Student D	3049

Q3. NOT EXISTS: sheet problems student 2 has not solved

```
SELECT sp.position, sp.problem_code
FROM sheet_problem sp
WHERE sp.sheet_id = 1
  AND NOT EXISTS (SELECT 1 FROM submission s
                  WHERE s.user_id = 2 AND s.problem_code = sp.problem_code
                    AND s.verdict = 'OK')
ORDER BY sp.position;
```

position	problem_code
1	2257D
2	2254E
3	2232C2
4	2229D

(4 of 8 rows shown)

Q5. Window function on a view: leaderboard

```
SELECT rank() OVER (ORDER BY solved DESC) AS rank, name, univ_name, solved
FROM v_leaderboard;
```

rank	name	univ_name	solved
1	Demo Student B	Amrita Vishwa Vidyapeetham	4596

rank	name	univ_name	solved
2	Demo Student C	Amrita Vishwa Vidyapeetham	3956
3	Demo Student D	NIT Calicut	3049
4	Demo Student A	Amrita Vishwa Vidyapeetham	1542

Q10. INTERSECT: problems solved by both student 1 and 2

```
SELECT p.problem_code, p.title
FROM problem p
WHERE p.problem_code IN (
    SELECT problem_code FROM submission WHERE user_id = 1 AND verdict = 'OK'
    INTERSECT
    SELECT problem_code FROM submission WHERE user_id = 2 AND verdict = 'OK')
ORDER BY p.rating DESC NULLS LAST
LIMIT 5;
```

problem_code	title
1523F	Favorite Game
1782G	Diverse Coloring
1446F	Line Distance
1349D	Slime and Biscuits

(4 of 5 rows shown)

Q6. Team analytics (CTE + window function)

For streaks we number each member's active days with row_number(). For days in a row, day − row_number() stays the same, so grouping by it gives each streak. Part of the output:

name	rating	solved	solved_30d	longest_streak	acceptance	unique
Demo Student A	2624	1542	7	7	50	676
Demo Student B	3353	4596	15	7	64	2849
Demo Student C	3676	3956	7	20	56	2396

8. Testing & Validation

We tested the database rules directly in SQL, and the app through its API. Database tests were rolled back after running.

8.1 Database tests

Table 5: Database tests

ID	Input	Expected	Actual output	Result
T1	Insert user with an e-mail that exists	Error: UNIQUE(email)	Rejected: duplicate key "app_user_email_key"	Pass
T2	Insert user with a handle that exists	Error: UNIQUE(cf_handle)	Rejected: duplicate key "app_user_cf_handle_key"	Pass
T3	Insert user with univ_id = 999	Error: foreign key	Rejected: "app_user" breaks foreign key constraint "app_user_univ_id_fkey"	Pass
T4	Insert user without a name	Error: NOT NULL	Rejected: null value in column "name" of relation "app_user" breaks not-null con	Pass
T5	Insert problem with rating 500	Error: CHECK rating 800–3500	Rejected: "problem" breaks check constraint "problem_rating_check"	Pass
T6	Insert submission with verdict 'ACCEPTED'	Error: CHECK verdict	Rejected: "submission" breaks check constraint "submission_verdict_check"	Pass
T7	Insert the same submission twice	Error: composite primary key	Rejected: duplicate key "submission_pkey"	Pass
T8	Submission for problem 99999Z	Error: foreign key	Rejected: "submission" breaks foreign key constraint "submission_problem_code_f	Pass
T9	4th member joins team 1	Rejected by trigger team_size_check	Rejected: team 1 already has 3 members	Pass
T10	Same problem twice in a sheet	Error: primary key (sheet_id, problem_code)	Rejected: duplicate key "sheet_problem_pkey"	Pass
T11	Two problems at position 1	Error: UNIQUE(sheet_id, position)	Rejected: duplicate key "sheet_problem_sheet_id_position_key"	Pass
T13	Team name 'O(1)' again	Error: UNIQUE(team_name)	Rejected: duplicate key "team_team_name_key"	Pass
T14	DELETE a sheet	Works; 0 sheet_problem rows left	Accepted; sheet_problem_rows = 0	Pass
T15	DELETE a team	Works; users remain with team_id = NULL	Accepted; users_left = 4, users_with_team = 0	Pass
T17	Problem with NULL rating	Works; difficulty NULL	Accepted; difficulty = NULL	Pass

8.2 Application tests

Table 6: Application tests

ID	Input	Expected	Actual	Result
A1	Log in with the wrong password	401, "Wrong password"	401, "Wrong password"	Pass
A3	Sign up with a 5-character password	400, password too short	400, "Password must be at least 8 characters"	Pass
A4	Sign up at a new college without its city	400, asks for the city	400, "This is a new college: please enter its city"	Pass
A5	Sign up with an e-mail already registered	409, e-mail already registered	409, "That e-mail is already registered"	Pass
A6	Open the dashboard without signing in	401, "Please sign in"	401, "Please sign in"	Pass
A7	Create a sheet with a code that is not on Codeforces	400, not a Codeforces problem	400, "99999Z is not a Codeforces problem"	Pass
A8	Create a sheet with no problems	400, needs at least one problem	400, "A sheet needs a title and at least one problem"	Pass
A9	Create a second team while already in one	400, leave current team first	400, "Leave your current team first"	Pass

All 15 database tests and 8 application tests passed. We also checked every page on a laptop and a phone screen, and made sure buttons cannot be double-clicked to send two requests.

9. Results & Screenshots

Screenshots of the working app with our demo data:

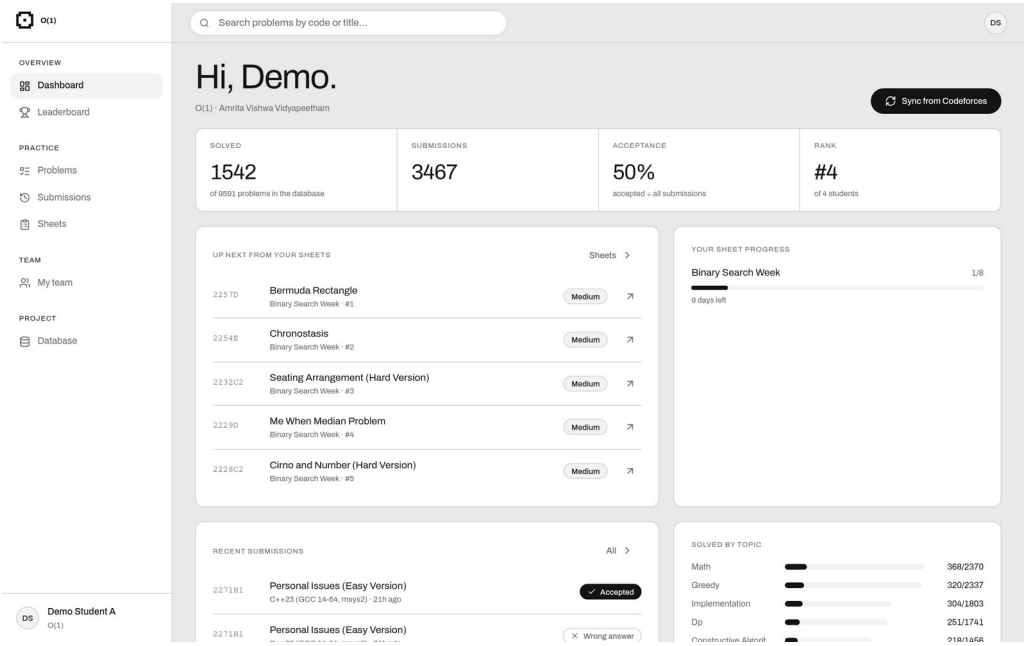


Figure 4: Dashboard: solved count, acceptance and rank come from one query; "Up next" uses NOT EXISTS.

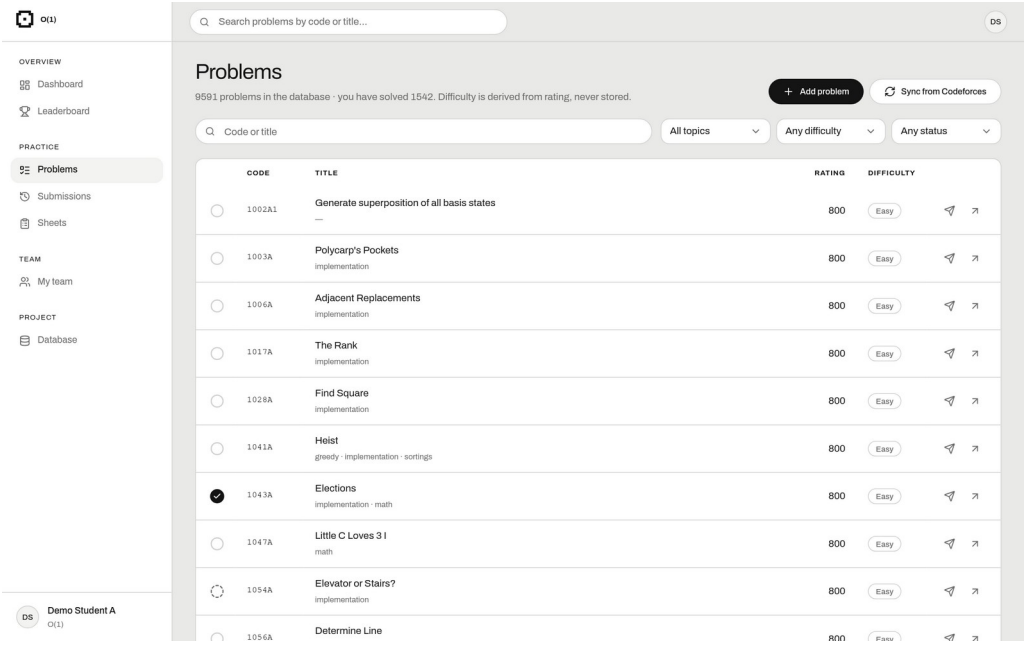


Figure 5: Problems: every stored problem with its topics, difficulty (from the view) and my status.

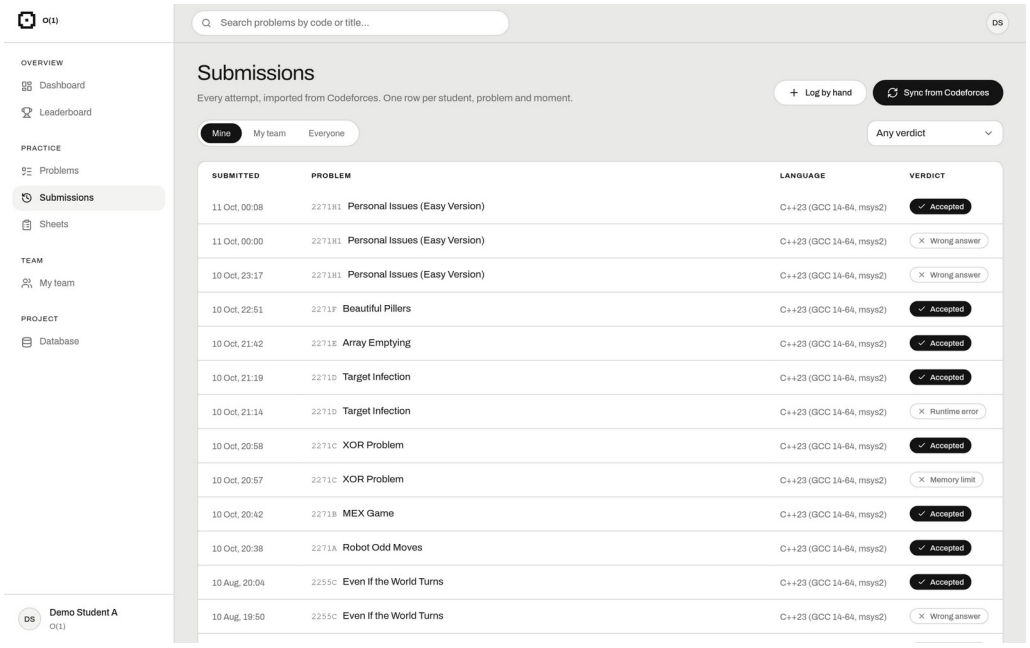


Figure 6: Submissions: a join of submission, app_user and problem, filtered by verdict.

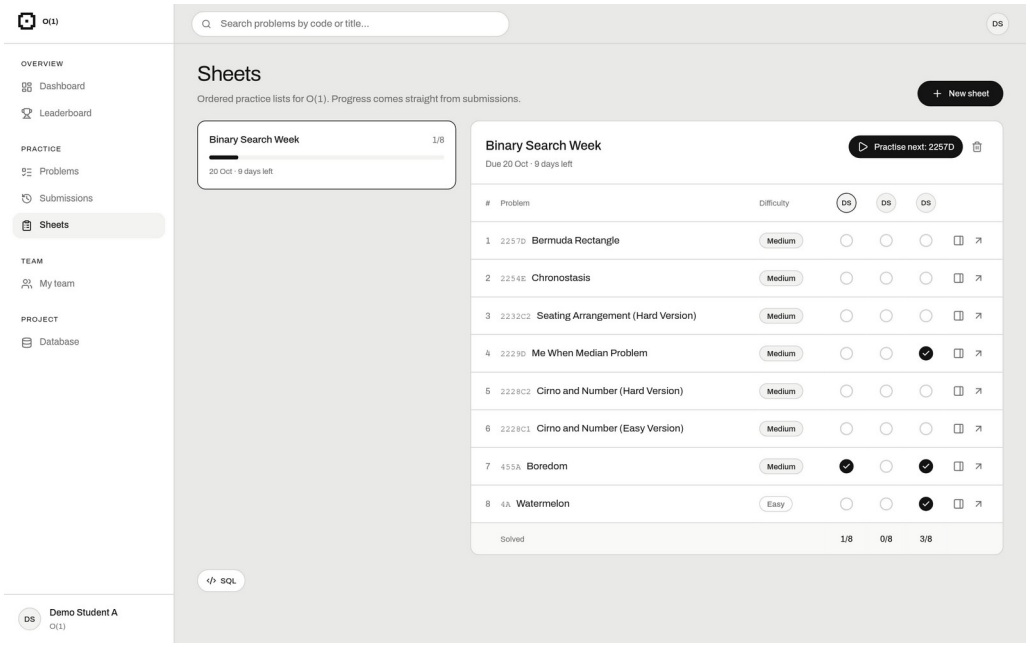


Figure 7: Team sheet: ticks show who solved each problem; progress comes from the v_sheet_progress view.

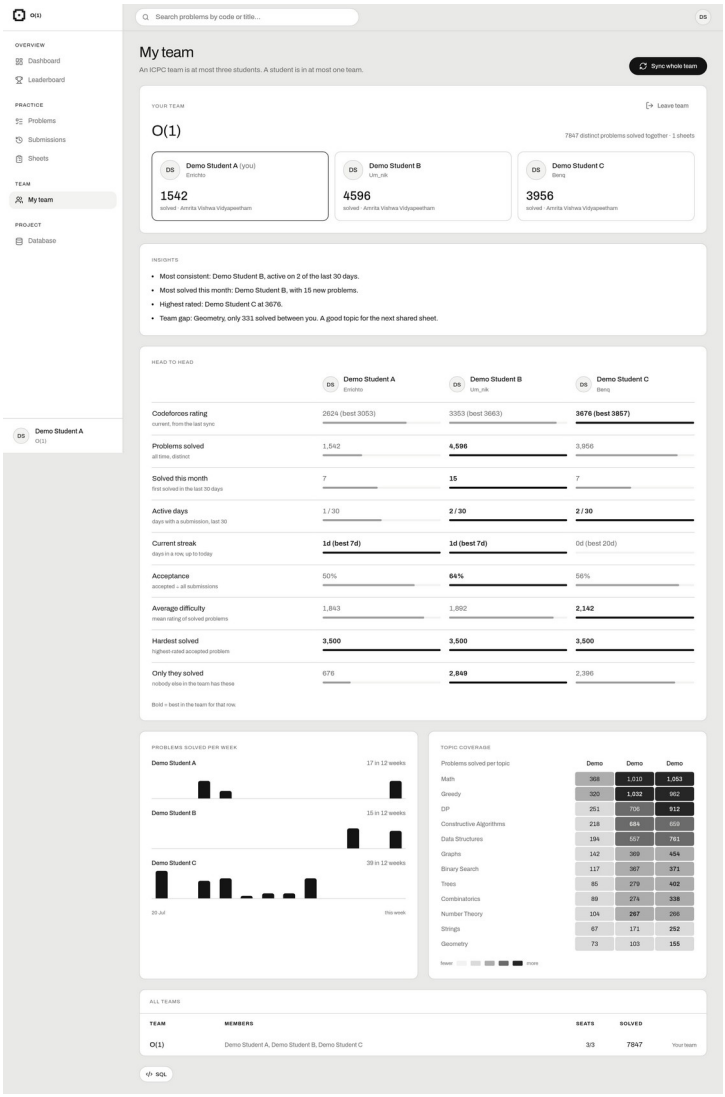


Figure 8: Team analytics: side-by-side numbers, weekly activity and topic heatmap.

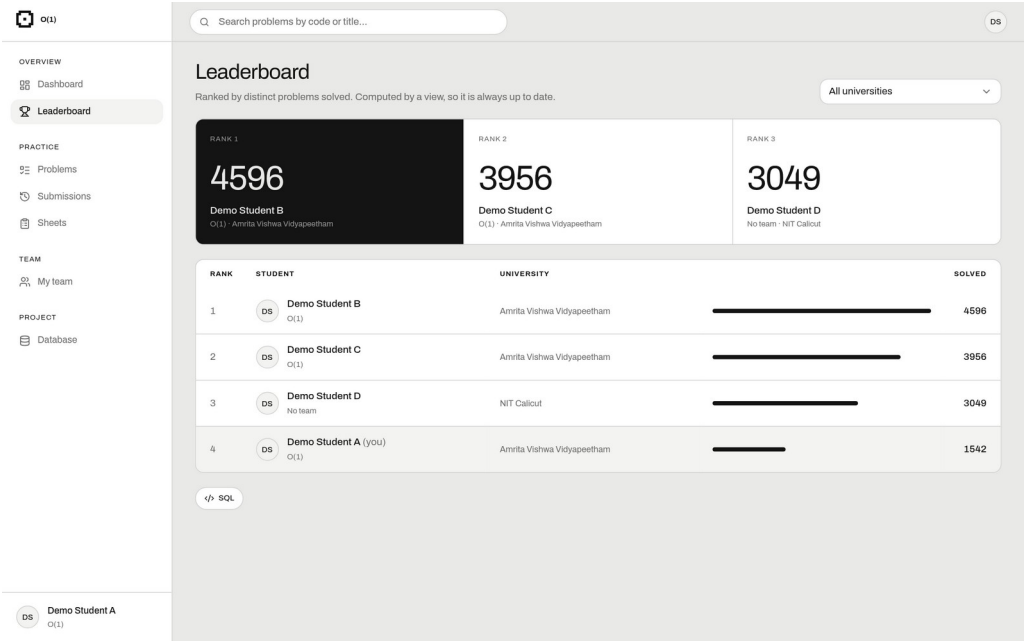


Figure 9: Leaderboard: v_leaderboard ranked with rank() OVER.

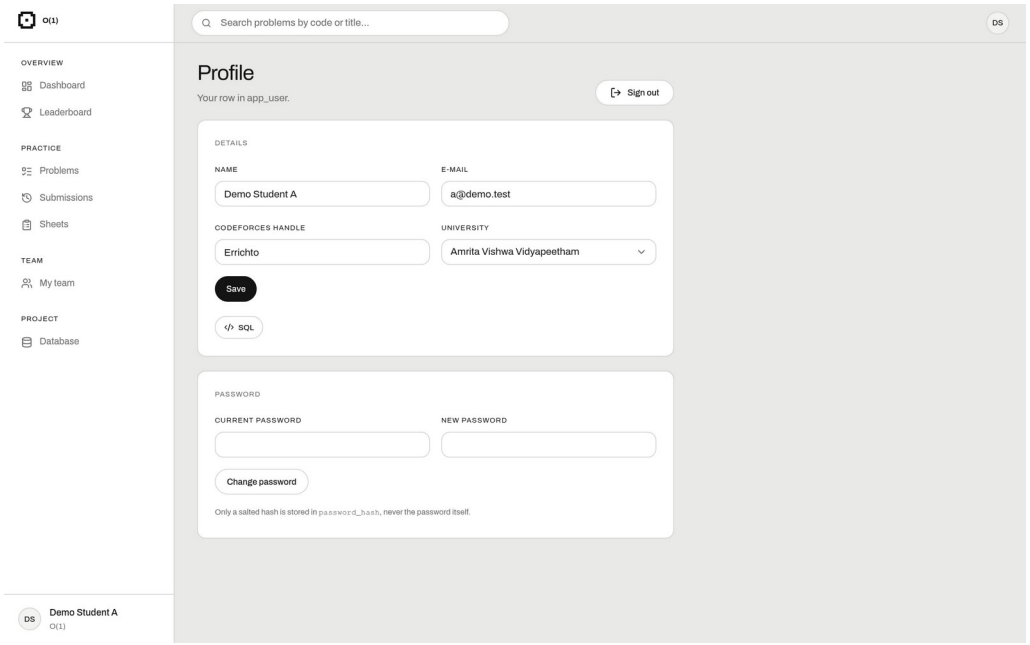


Figure 10: Profile: updates app_user; the database rejects an e-mail or handle that is already used.

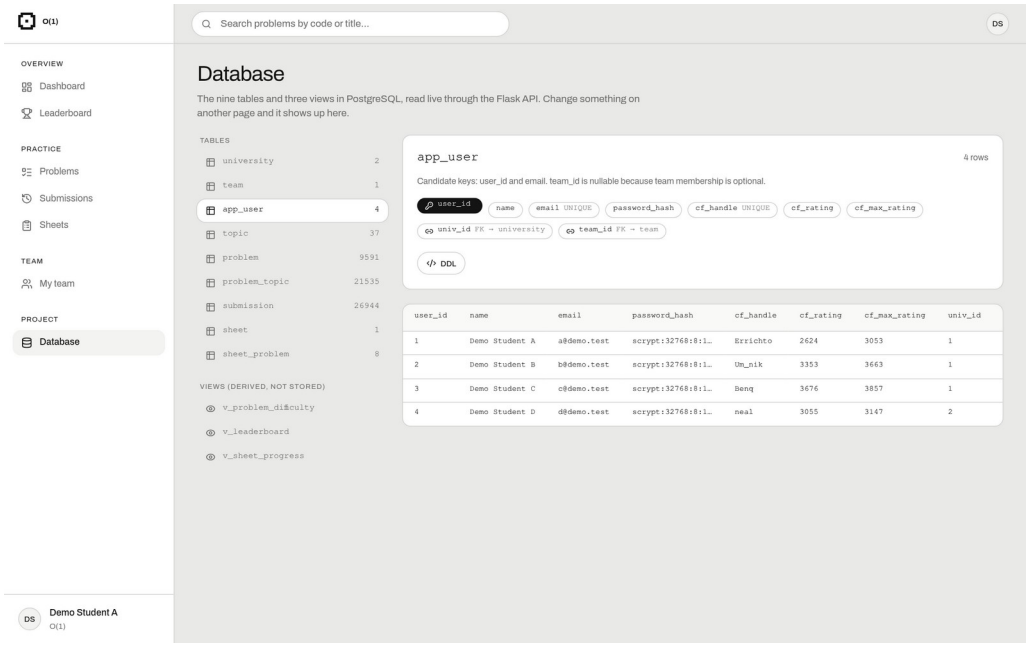


Figure 11: Database page: live tables with keys, DDL and rows.

10. Challenges Faced & Solutions

Table 7: Challenges and solutions

Challenge	How we solved it
Our first design had 20 tables and was too hard to explain.	We cut it to 9 tables and kept only the core features.
A problem has many topics, which broke 1NF.	Made a Topic table and the problem_topic junction table.
Stored counts and difficulty could go out of date.	We calculate them with views instead of storing them.
"Max 3 members" cannot be a CHECK constraint.	Wrote a trigger that counts members before insert or update.
Students typed the same college in different ways.	Used pg_trgm fuzzy search to suggest the existing name.
Syncing twice made duplicates; Codeforces limits API calls.	Composite primary key with ON CONFLICT DO NOTHING; calls are spaced 2 seconds apart.
Password hashes were too long for VARCHAR(100).	Changed the column to VARCHAR(255).
Creating a team takes two statements.	One transaction per request, so both happen or neither does.
Calculating streaks in SQL.	Used the "gaps and islands" trick with row_number().
Codeforces cannot be shown inside our app.	Problems open in a new tab instead.

11. Conclusion & Future Enhancements

We replaced a messy spreadsheet with a nine-table PostgreSQL database in BCNF. We designed it step by step using FDs, closures, a minimal cover and normalisation, and showed it is lossless and dependency preserving. The database itself enforces the rules with keys, constraints and a trigger, and views keep calculated values correct. On top of it we built a working web app with Codeforces import, teams, shared sheets, a leaderboard and team analytics. All our objectives were met and all tests passed.

Future work: contests and rating history, a coach role, more judges (AtCoder, CodeChef), automatic daily sync, password reset by e-mail, and hosting the app online.

References

1. Silberschatz, Korth and Sudarshan, Database System Concepts, 7th ed., McGraw-Hill, 2019.
2. Elmasri and Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.
3. PostgreSQL 18 Documentation (including pg_trgm), https://www.postgresql.org/docs/
4. Codeforces API, https://codeforces.com/apiHelp
5. Flask documentation, https://flask.palletsprojects.com/ and Psycopg 2, https://www.psycopg.org/docs/
6. React documentation, https://react.dev/